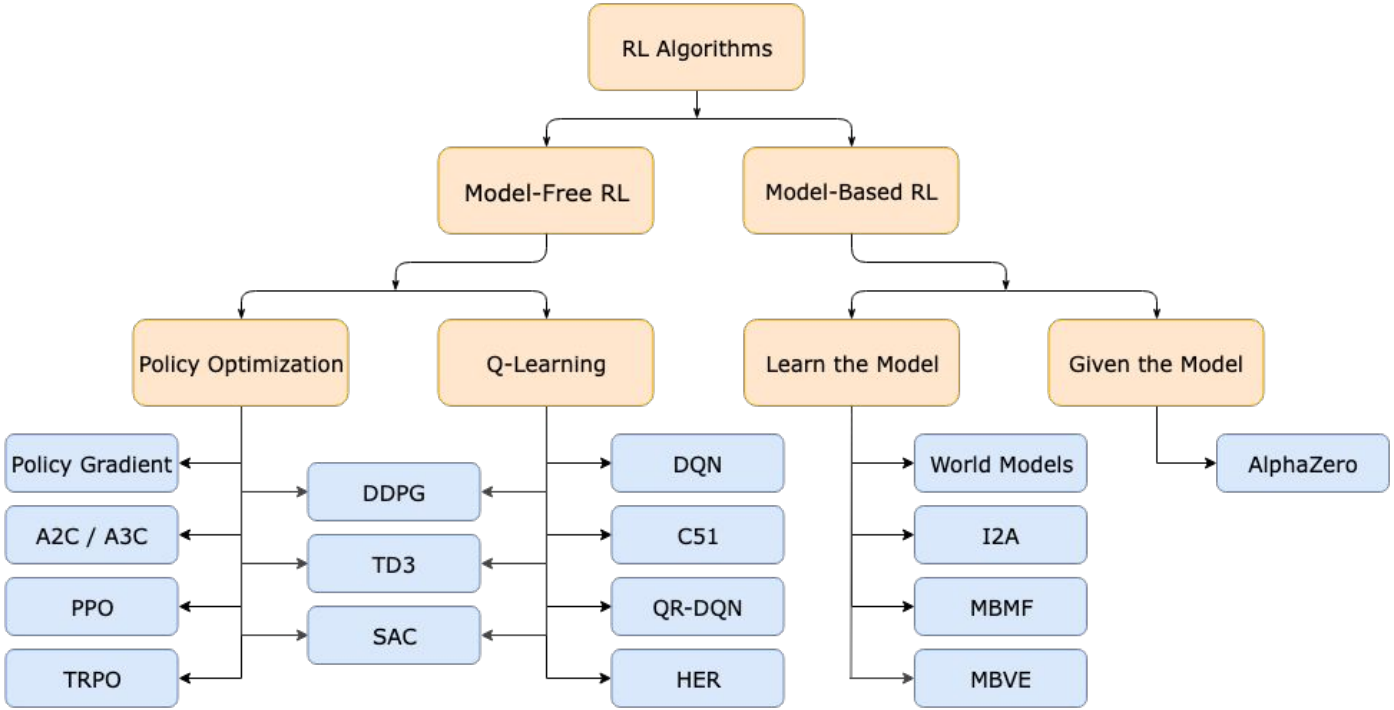
Deep Reinforcement Learning

Genya Ishigaki

Types of RL Algorithms

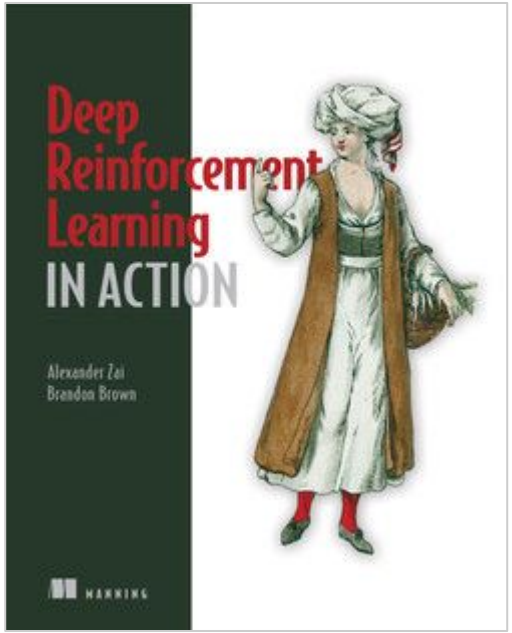


https://spinningup.openai.com/en/latest/spinningup/rl_intro2.html

Reference

Deep Reinforcement Learning in Action
by Alexander Zai, Brandon Brown
March 2020 ISBN: 9781617295430

Login to Safari Books Online (O'Reilly) with your SJSU account [Free!]
https://learning.oreilly.com/home/



Historical Remarks

- In 2013, DeepMind “**Playing Atari with Deep Reinforcement Learning**”
- Their new approach to an old algorithm (Q-Learning), which gave them enough performance to play six of seven Atari 2600 games at record levels
- The general boost that deep learning (NNs) got a few years prior with the use of GPUs that allowed the training of much larger networks
- Other implementation techniques they invented to solve RL-unique issues

Playing Atari with Deep Reinforcement Learning

Volodymyr Mnih **Koray Kavukcuoglu** **David Silver** **Alex Graves** **Ioannis Antonoglou**
Daan Wierstra **Martin Riedmiller**
DeepMind Technologies
{vlad,koray,david,alex.graves,ioannis,daan,martin.riedmiller} @ deepmind.com

Abstract

We present the first deep learning model to successfully learn control policies directly from high-dimensional sensory input using reinforcement learning. The model is a convolutional neural network, trained with a variant of Q-learning, whose input is raw pixels and whose output is a value function estimating future rewards. We apply our method to seven Atari 2600 games from the Arcade Learning Environment, with no adjustment of the architecture or learning algorithm. We find that it outperforms all previous approaches on six of the games and surpasses a human expert on three of them.

1 Introduction

Learning to control agents directly from high-dimensional sensory inputs like vision and speech is one of the long-standing challenges of reinforcement learning (RL). Most successful RL applications that operate on these domains have relied on hand-crafted features combined with linear value functions or policy representations. Clearly, the performance of such systems heavily relies on the quality of the feature representation.

Recent advances in deep learning have made it possible to extract high-level features from raw sensory data, leading to breakthroughs in computer vision [11, 22, 16] and speech recognition [6, 7]. These methods utilise a range of neural network architectures, including convolutional networks, multilayer perceptrons, restricted Boltzmann machines and recurrent neural networks, and have exploited both supervised and unsupervised learning. It seems natural to ask whether similar techniques could also be beneficial for RL with sensory data.

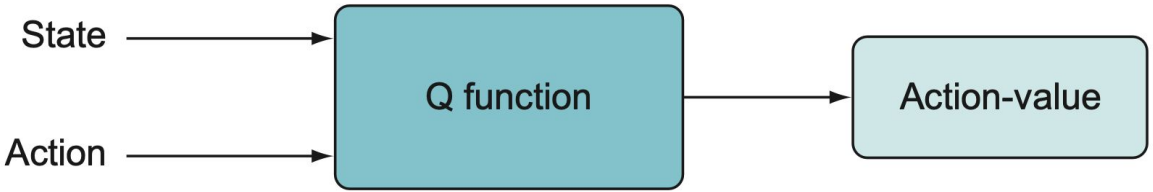
However reinforcement learning presents several challenges from a deep learning perspective. Firstly, most successful deep learning applications to date have required large amounts of hand-labelled training data. RL algorithms, on the other hand, must be able to learn from a scalar reward signal that is frequently sparse, noisy and delayed. The delay between actions and resulting rewards, which can be thousands of timesteps long, seems particularly daunting when compared to the direct association between inputs and targets found in supervised learning. Another issue is that most deep learning algorithms assume the data samples to be independent, while in reinforcement learning one typically encounters sequences of highly correlated states. Furthermore, in RL the data distribution changes as the algorithm learns new behaviours, which can be problematic for deep learning methods that assume a fixed underlying distribution.

This paper demonstrates that a convolutional neural network can overcome these challenges to learn successful control policies from raw video data in complex RL environments. The network is trained with a variant of the Q-learning [26] algorithm, with stochastic gradient descent to update the weights. To alleviate the problems of correlated data and non-stationary distributions, we use

1

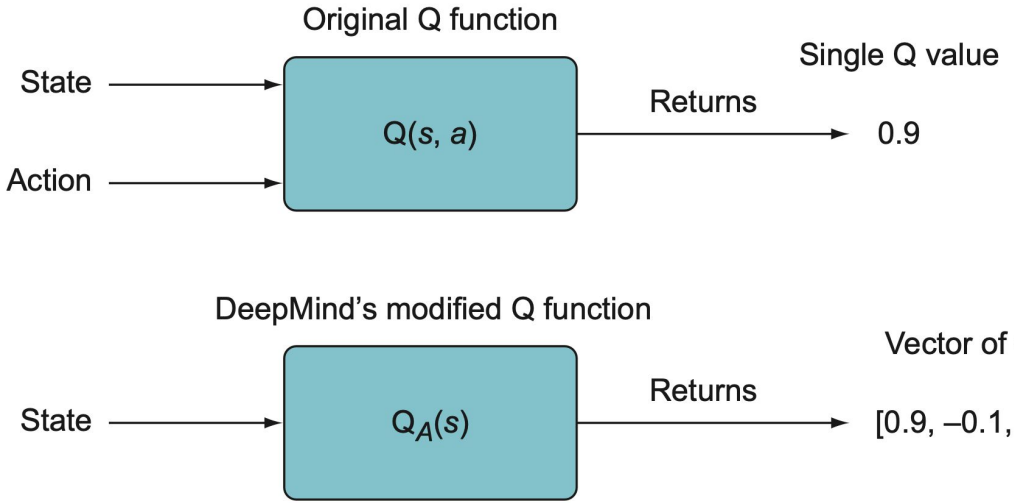
Q Function Approximation

- The Q function could be any function that
  - Accepts a state and action
  - Returns the value of taking that action given that state
- As we saw in the last lecture, a neural network can be used as a function approximator



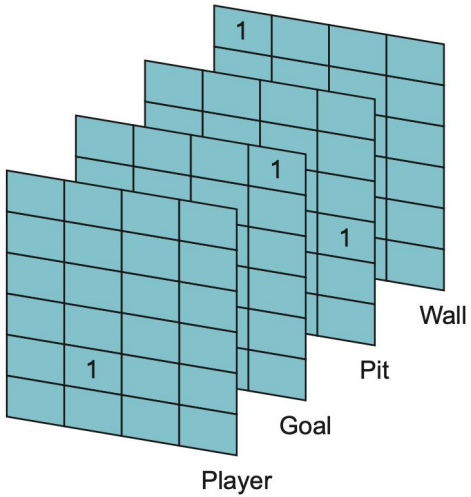
Point 1. Design of NN-based Q Function

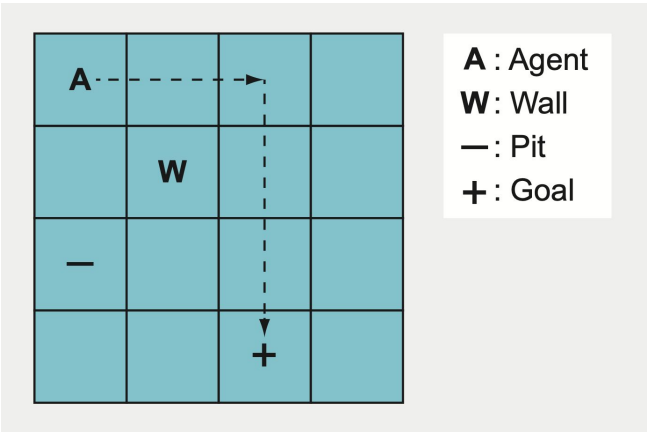
- In DQN, the Q function is implemented in a way to compute the Q values for all actions, given a state
- The vector-valued Q function is more efficient since you only need to compute the function once for all the actions



An Example: Grid World

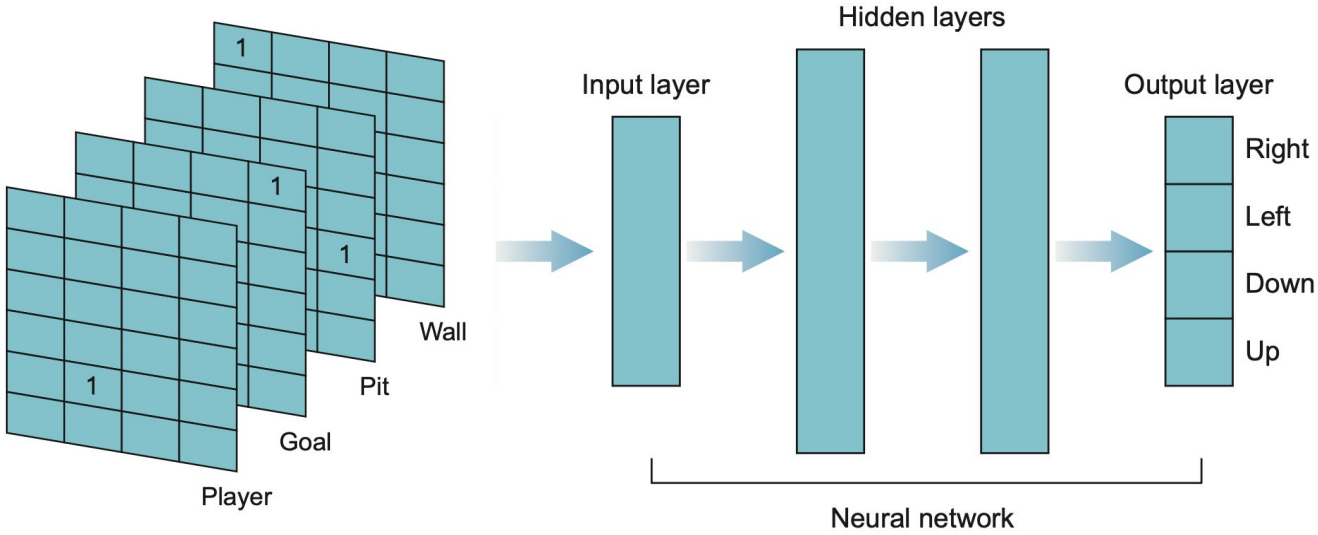
- The agent (A) must navigate along the shortest path to the goal tile (+) and avoid falling into the pit (–)
- Here, we use a 4 × 4 × 4 tensor to represent a state, where each frame (4 x 4) indidates
  - The agent
  - The goal
  - The pit
  - The wall





A neural network as the Q function

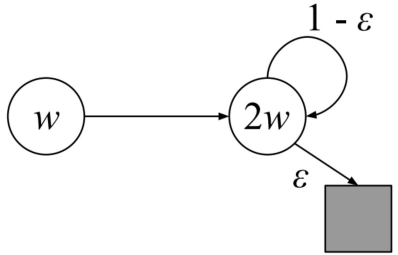
- An input layer that can accept a 64-length game state vector (4 x 4 x 4)
- Some hidden layers
- An output layer that produces a 4-length vector of Q values for each action



Tsitsiklis and Van Roy's Counterexample

- Each state is described by a single scalar feature φ
  - φ(s1) = 1 and φ(s2) = 2
- The value function is approximated with a linear function v(s, w) = w × φ(s)
- All rewards are 0s
  - The true value function gives ZERO to all states (w* = 0)
- Theoretically, the following DP should represent the optimal updates

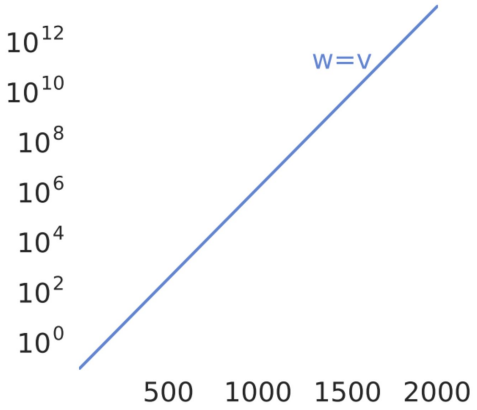
$$
\begin{aligned}
w_{k+1} &= \operatorname*{arg\,min}_{w \in \mathbb{R}} \sum_{s \in \mathcal{S}} \Big(\hat{v}(s,w) - \mathbb{E}_{\pi}\big[R_{t+1} + \gamma \hat{v}(S_{t+1}, w_k) \mid S_t = s \big] \Big)^2 \\
&= \operatorname*{arg\,min}_{w \in \mathbb{R}} \big(w - \gamma 2 w_k \big)^2 + \big(2w - (1-\varepsilon)\gamma 2 w_k \big)^2 \\
&= \frac{6 - 4\varepsilon}{5} \gamma w_k .
\end{aligned}
$$

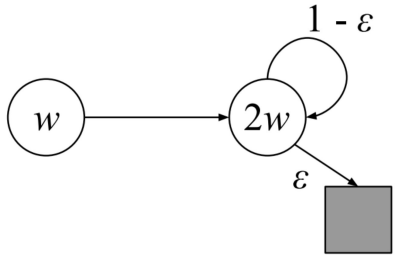


Note: This example shows that linear function approximation would not work with DP even if the least-squares solution was found at each step. The divergence problem is a universal issue with the approximation methods, including Deep RL.

Tsitsiklis and Van Roy's Counterexample

- When $\gamma > \frac{5}{6 - 4\varepsilon}$ and $w_0 \neq 0$, the sequence of update diverges.
- That seems quite possible (e.g. gamma = 0.89 with epsilon = 0.1)
- This example shows that the approximation methods bring instability in the learning process with particular settings

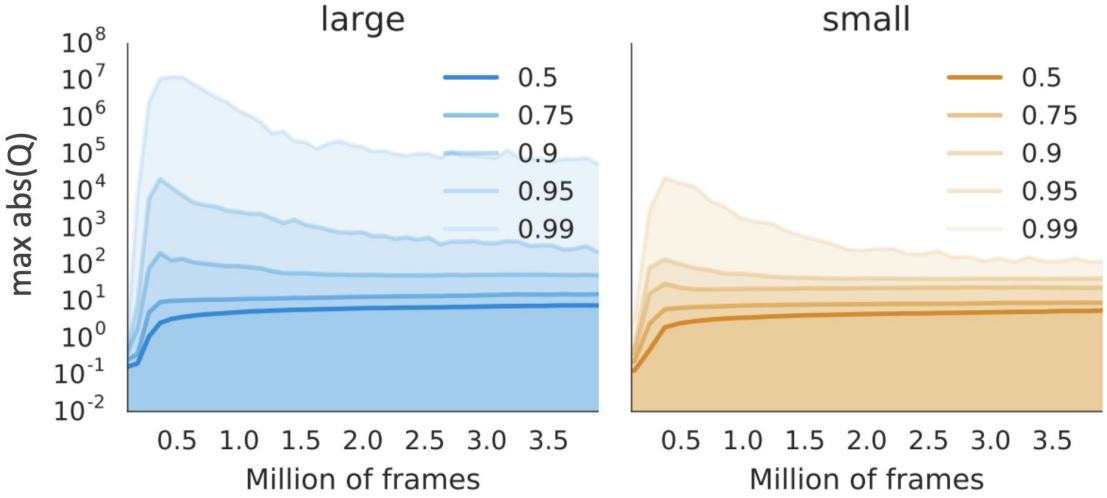




Note: This example shows that linear function approximation would not work with DP even if the least-squares solution was found at each step. The divergence problem is a universal issue with the approximation methods, including Deep RL.

The Divergence Issue with Deep RL

- Empirically, we actually find that unbounded divergence is rare with Deep RL
- More common are value explosions that recover after an initial phase
- This phenomenon is also referred to as “soft-divergence”



Van Hasselt, Hado, et al. "Deep reinforcement learning and the deadly triad." arXiv preprint arXiv:1812.02648 (2018).

The Deadly Triad

- **Function approximation:** A powerful, scalable way of generalizing from a state space much larger than the memory and computational resources (e.g., linear function approximation or ANNs)
- **Bootstrapping:** Update targets that include existing estimates (as in dynamic programming or TD methods) rather than relying exclusively on actual rewards and complete returns (as in MC methods)
- **Off-policy training:** Training on a distribution of transitions other than that produced by the target policy. Sweeping through the state space and updating all states uniformly, as in dynamic programming, does not respect the target policy and is an example of off-policy training

The Deadly Triad

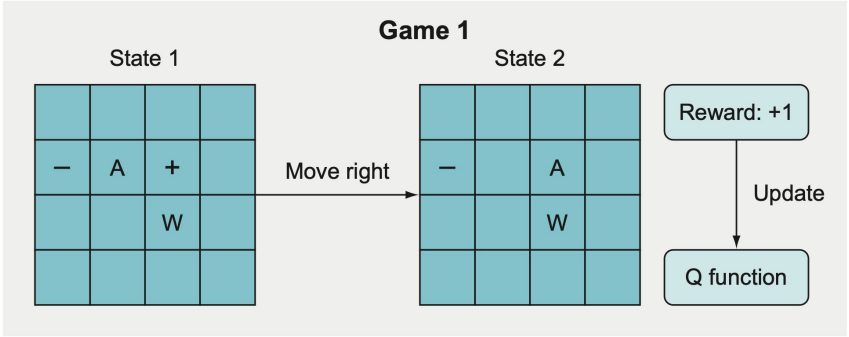
- The instability and divergence are
  - NOT due to control or to generalized policy iteration (The issue happens even with the simpler prediction task)
  - NOT due to learning or to uncertainties about the environment (The issue occurs just as strongly in planning methods (DP) in which the environment is completely known)
- If **any two elements of the deadly triad** are present, then instability can be avoided
- It is natural, to go through the three and see if there is any one that can be given up

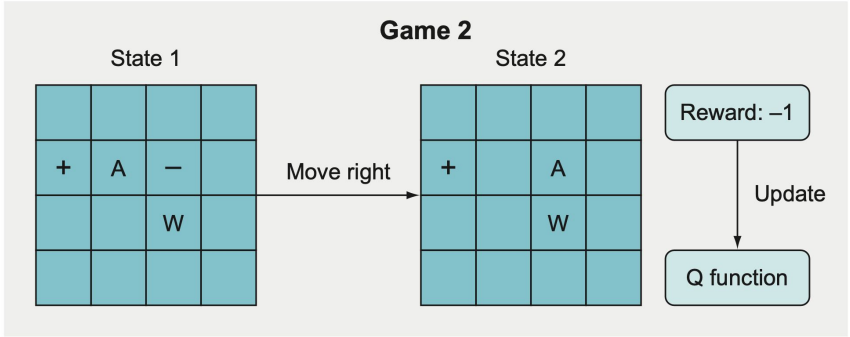
The Deadly Triad

- **Function approximation:**
  - It is impossible to give this up for the scalability!
- **Bootstrapping:**
  - Maybe, but the data efficiency will be sacrificed by going back to MC
  - Also, MC is computationally expensive!
- **Off-policy training:**
  - If you can use SARSA instead of Q-learning, go for it!
  - But in many practical problems, we use samples collected by other agents with other policies
- In conclusion, it is not always possible to give up these things in many practical situations
- We need to solve the divergence issue with another technique

Another Issue: Catastrophic Forgetting

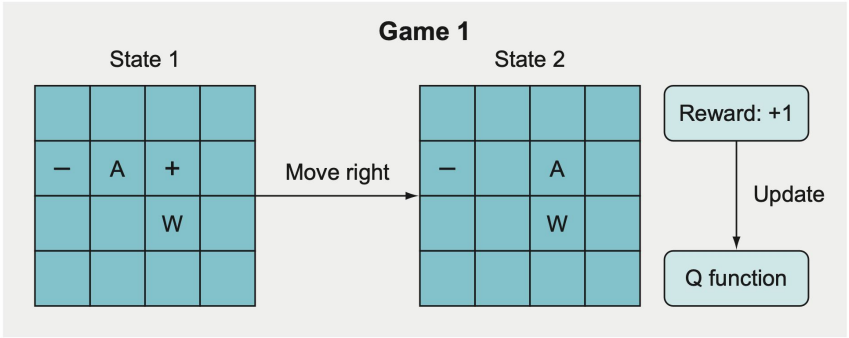
- The divergence issue is a common problem among the approximation methods
- The catastrophic forgetting is associated with gradient descent-based training methods in online training
- e.g. Assume that we generate different settings of the Grid World game and train the DQN agent

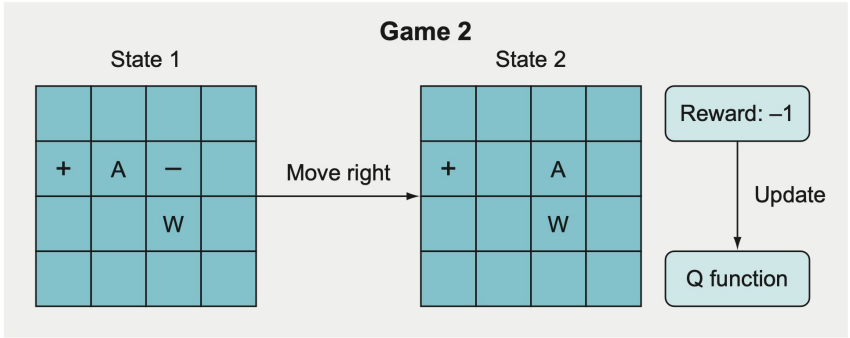




Another Issue: Catastrophic Forgetting

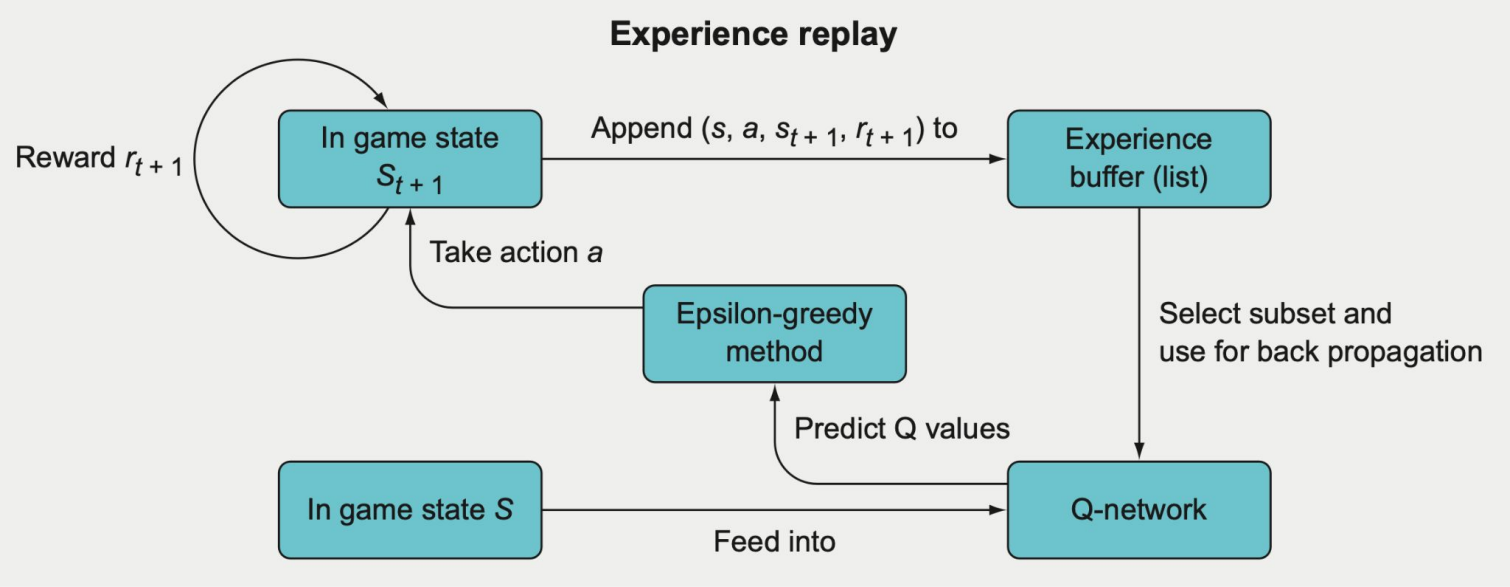
- Game 1
  - The agent takes RIGHT action and receives a positive reward
- Game 2
  - The agent takes RIGHT action and receives a negative reward
- A push-pull between very similar state-actions (but with divergent targets) that results in this inability to properly learn anything





Point 2. Experience Replay

- Instead of discarding a single experience after the gradient update,
- Store multiple experiences in a replay buffer
- Update the weights based on experiences sampled from the buffer



Point 2. Experience Replay

Algorithm 1 Deep Q-learning with Experience Replay

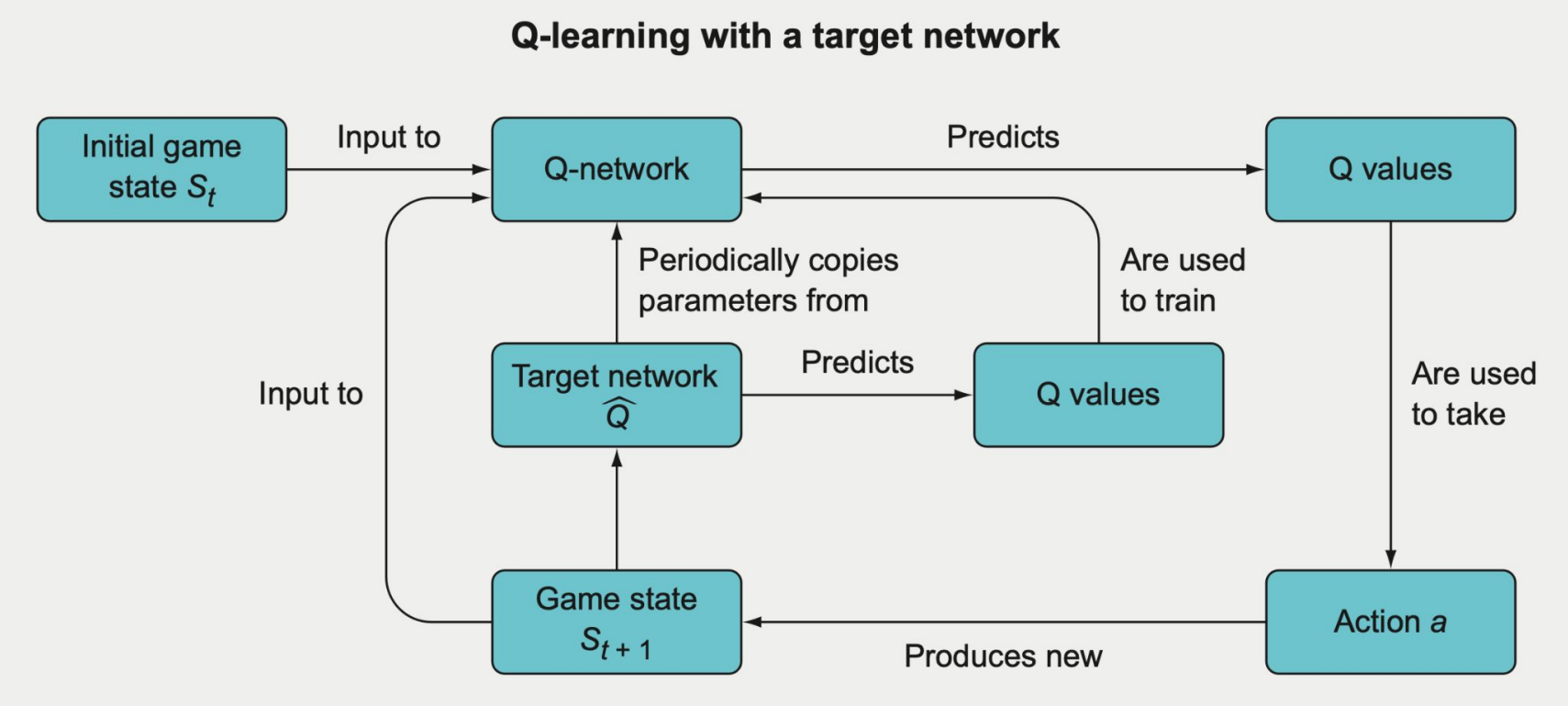
Initialize replay memory $\mathcal{D}$ to capacity N
Initialize action-value function Q with random weights
for episode $= 1, M$ **do**
  Initialise sequence $s_1 = \{x_1\}$ and preprocessed sequenced $\phi_1 = \phi(s_1)$
  for $t = 1, T$ **do**
    With probability ϵ select a random action a_t
    otherwise select $a_t = \max_a Q^*(\phi(s_t), a; \theta)$
    Execute action a_t in emulator and observe reward r_t and image x_{t+1}
    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
    Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$
    Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$
    Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$
    Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$
  end for
end for

Point 3. Target Net

- We can address this in our deep RL agents using a freezed target network
  - Hold the parameters used to compute the bootstrap targets
  - Only update them periodically (every few hundreds or thousands of updates)
- **More stable** targets to compute the TD error
- This breaks the feedback loop that sits at the heart of the deadly trial
- Also, we can do soft update

$$\theta^{Q'} \leftarrow \tau\theta^{Q} + (1-\tau)\theta^{Q'}$$

Point 3. Target Net



Further Improvement: Deep Double Q-learning

- Q-learning has an overestimation bias, that can be corrected by double Q-learning

$$J(\theta) = \frac{1}{2}\left(R_{i+1} + \gamma q\left(S_{i+1}, \underset{a}{\text{argmax}} q(S_{i+1}, a, \theta), \theta^{-}\right) - q(S_i, A_i, \theta)\right)^2$$

- Great combination with target networks
  - We can use the frozen params as θ^{-}

Further Improvement: Prioritized Replay

- DQN samples uniformly from the replay buffer
- Idea: Prioritize transitions on which we can learn much
- Basic implementation:
  - Priority of sample i = (TD error)

$$R_{i+1} + \gamma q\left(S_{i+1}, \underset{a}{\operatorname{argmax}} q(S_{i+1}, a, \theta), \theta^{-}\right) - q(S_i, A_i, \theta)$$

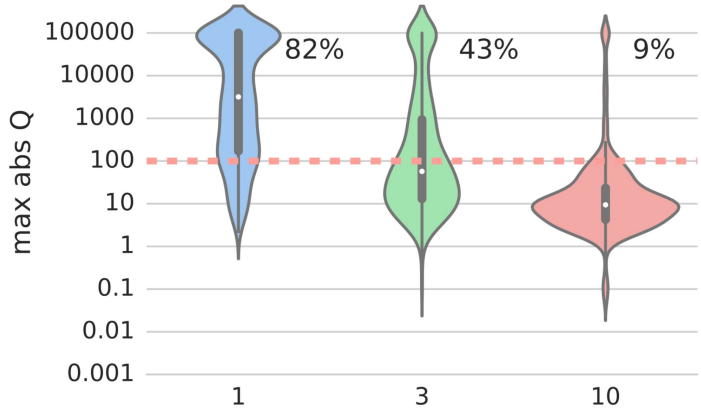
- Sample according to priority

Further Improvement: Multi-step Control

- Define the n-step Q-learning target

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \ldots + \gamma^{n-1} R_{t+n} + \gamma^n \underbrace{q\left(S_{i+1}, \operatorname{argmax}\ q\left(S_{i+1}, a, \theta\right), \theta^-\right)}_{\text{Double Q bootstrap target}}$$

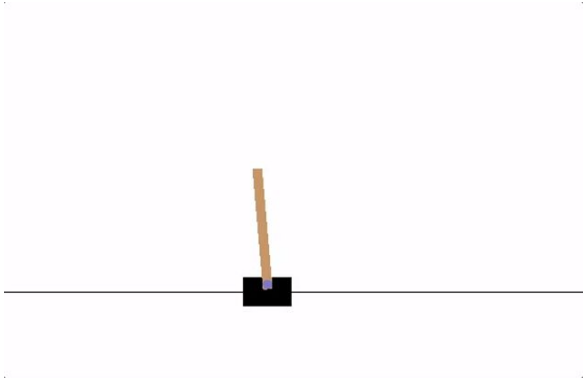
- The Return is partially on-policy and the bootstrapping is off-policy
- A well-defined target: “On-policy for n steps, and then act greedy”
- Multi-step targets allow to trade-off bias and variance



Van Hasselt, Hado, et al. "Deep reinforcement learning and the deadly triad." arXiv preprint arXiv:1812.02648 (2018).

Implementational Aspects of DQN

- Implement the DQN with PyTorch to train the agent on the CartPole-v1 task
- What is PyTorch?
  - Defining neural networks and related components like activation functions
  - Providing different types of optimizers
  - Computing gradients automatically (autograd)
  - Supporting GPU computations



https://pytorch.org/tutorials/intermediate/reinforcement_q_learning.html

Intro to PyTorch: Autograd

- **torch.autograd** is PyTorch's automatic differentiation engine
- Conceptually, autograd records a computation graph of all operations that created the data as you execute operations (forward pass)
- By tracing the recorded graph, autograd can automatically compute the gradients using the chain rule
- Steps:
  1. Define tensors (NN prams) with `requires_grad=True`
  2. Define the loss function Q in terms of the tensors
  3. By calling `Q.backward()`, autograd calculates the gradients of all tensors and stores them in the respective tensors' `.grad` attribute

https://pytorch.org/docs/stable/notes/autograd.html
https://pytorch.org/tutorials/beginner/blitz/autograd_tutorial.html

Intro to PyTorch: Autograd

- **`Q.backward(gradient)`**
  - The gradient of the function being differentiated w.r.t. self.
  - This argument can be omitted if self is a scalar.

```
1  import torch
2
3  # "requires_grad=True" signals to autograd that every
   operation on them should be tracked.
4  a = torch.tensor([2., 3.], requires_grad=True)
5  b = torch.tensor([6., 4.], requires_grad=True)
6
7  Q = 3*a**3 - b**2
8
9  external_grad = torch.tensor([1., 1.])
10 Q.backward(gradient=external_grad)
11
12 print(a.grad)
```

$$Q = 3a^3 - b^2$$

$$\frac{\partial Q}{\partial a} = 9a^2$$

$$\frac{\partial Q}{\partial b} = -2b$$

Replay Buffer

```
Transition = namedtuple('Transition',
                        ('state', 'action', 'next_state', 'reward'))


class ReplayMemory(object):

    def __init__(self, capacity):
        self.memory = deque([], maxlen=capacity)

    def push(self, *args):
        """Save a transition"""
        self.memory.append(Transition(*args))

    def sample(self, batch_size):
        return random.sample(self.memory, batch_size)

    def __len__(self):
        return len(self.memory)
```

- Transition - a named tuple representing a single transition in our environment
- ReplayMemory - a cyclic buffer of bounded size that holds the transitions observed recently

https://pytorch.org/tutorials/intermediate/reinforcement_q_learning.html

Q-network

```
class DQN(nn.Module):

    def __init__(self, n_observations, n_actions):
        super(DQN, self).__init__()
        self.layer1 = nn.Linear(n_observations, 128)
        self.layer2 = nn.Linear(128, 128)
        self.layer3 = nn.Linear(128, n_actions)

    # Called with either one element to determine next action, or a batch
    # during optimization. Returns tensor([[left0exp,right0exp]...]).
    def forward(self, x):
        x = F.relu(self.layer1(x))
        x = F.relu(self.layer2(x))
        return self.layer3(x)
```

- Define linear layers
- Input size = the size of an observation
- Output size = the size of the action space
- forward pass with relu activations

Utilities

```
# Get number of actions from gym action space
n_actions = env.action_space.n
# Get the number of state observations
state, info = env.reset()
n_observations = len(state)

policy_net = DQN(n_observations, n_actions).to(device)
target_net = DQN(n_observations, n_actions).to(device)
target_net.load_state_dict(policy_net.state_dict())

optimizer = optim.AdamW(policy_net.parameters(), lr=LR, amsgrad=True)
memory = ReplayMemory(10000)
```

- Policy and target nets
- Adam optimizer dynamically adjusts learning rates based on past gradients and their second moments

Utilities

```
def select_action(state):
    global steps_done
    sample = random.random()
    eps_threshold = EPS_END + (EPS_START - EPS_END) * \
        math.exp(-1. * steps_done / EPS_DECAY)
    steps_done += 1
    if sample > eps_threshold:
        with torch.no_grad():
            # t.max(1) will return the largest column value of each row.
            # second column on max result is index of where max element was
            # found, so we pick action with the larger expected reward.
            return policy_net(state).max(1).indices.view(1, 1)
    else:
        return torch.tensor([[env.action_space.sample()]], device=device, dtype=torch.long)
```

- The epsilon-greedy method
- t.max(1) returns the max value (exploitation)
- The “else” case returns a random result (exploration)

Training loop (1)

```
def optimize_model():
    if len(memory) < BATCH_SIZE:
        return
    transitions = memory.sample(BATCH_SIZE)
    # Transpose the batch (see https://stackoverflow.com/a/19343/3343043 for
    # detailed explanation). This converts batch-array of Transitions
    # to Transition of batch-arrays.
    batch = Transition(*zip(*transitions))

    # Compute a mask of non-final states and concatenate the batch elements
    # (a final state would've been the one after which simulation ended)
    non_final_mask = torch.tensor(tuple(map(lambda s: s is not None,
                                            batch.next_state)), device=device, dtype=torch.bool)
    non_final_next_states = torch.cat([s for s in batch.next_state
                                                if s is not None])
    state_batch = torch.cat(batch.state)
    action_batch = torch.cat(batch.action)
    reward_batch = torch.cat(batch.reward)

    # Compute Q(s_t, a) - the model computes Q(s_t), then we select the
    # columns of actions taken. These are the actions which would've been taken
    # for each batch state according to policy_net
    state_action_values = policy_net(state_batch).gather(1, action_batch)
```

- An optimize_model function that performs a single step of the optimization
- V(s) = 0 for terminal s

Training loop (2)

```
def optimize_model():
                                    ....
    # Compute V(s_{t+1}) for all next states.
    # Expected values of actions for non_final_next_states are computed based
    # on the "older" target_net; selecting their best reward with max(1).values
    # This is merged based on the mask, such that we'll have either the expected
    # state value or 0 in case the state was final.
    next_state_values = torch.zeros(BATCH_SIZE, device=device)
    with torch.no_grad():
        next_state_values[non_final_mask] = target_net(non_final_next_states).max(1).values
    # Compute the expected Q values
    expected_state_action_values = (next_state_values * GAMMA) + reward_batch

    # Compute Huber loss
    criterion = nn.SmoothL1Loss()
    loss = criterion(state_action_values, expected_state_action_values.unsqueeze(1))

    # Optimize the model
    optimizer.zero_grad()
    loss.backward()
    # In-place gradient clipping
    torch.nn.utils.clip_grad_value_(policy_net.parameters(), 100)
    optimizer.step()
```

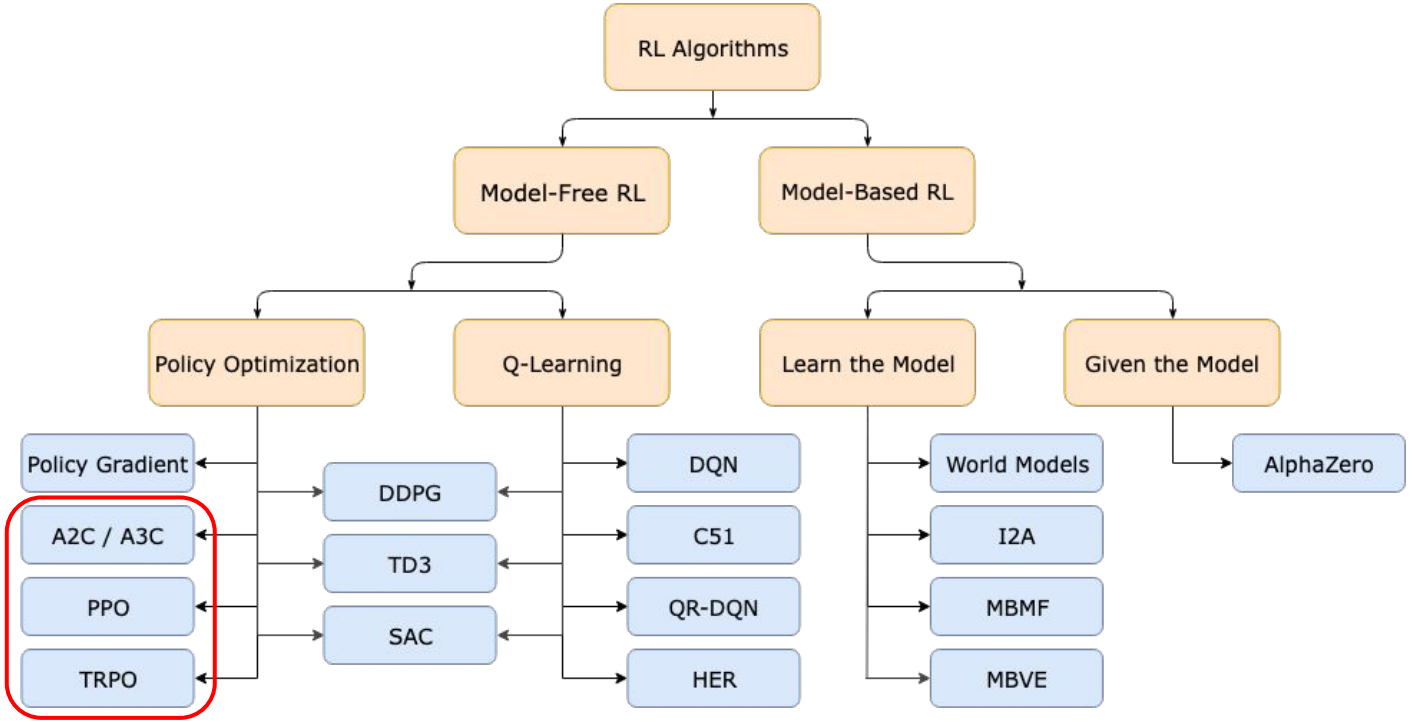
Outer loop for training (1)

```
for i_episode in range(num_episodes):
    # Initialize the environment and get its state
    state, info = env.reset()
    state = torch.tensor(state, dtype=torch.float32, device=device).unsqueeze(0)
    for t in count():
        action = select_action(state)
        observation, reward, terminated, truncated, _ = env.step(action.item())
        reward = torch.tensor([reward], device=device)
        done = terminated or truncated

        if terminated:
            next_state = None
        else:
            next_state = torch.tensor(observation, dtype=torch.float32,
device=device).unsqueeze(0)

        # Store the transition in memory
        memory.push(state, action, next_state, reward)

        # Move to the next state
        state = next_state

        # Perform one step of the optimization (on the policy network)
        optimize_model()
```

Outer loop for training (2)

```
for i_episode in range(num_episodes):
                                    ....
        # Soft update of the target network's weights
        # θ′ ← τ θ + (1 −τ )θ′
        target_net_state_dict = target_net.state_dict()
        policy_net_state_dict = policy_net.state_dict()
        for key in policy_net_state_dict:
            target_net_state_dict[key] = policy_net_state_dict[key]*TAU +
target_net_state_dict[key]*(1-TAU)
        target_net.load_state_dict(target_net_state_dict)

        if done:
            episode_durations.append(t + 1)
            plot_durations()
            break
```

Types of RL Algorithms



https://spinningup.openai.com/en/latest/spinningup/rl_intro2.html

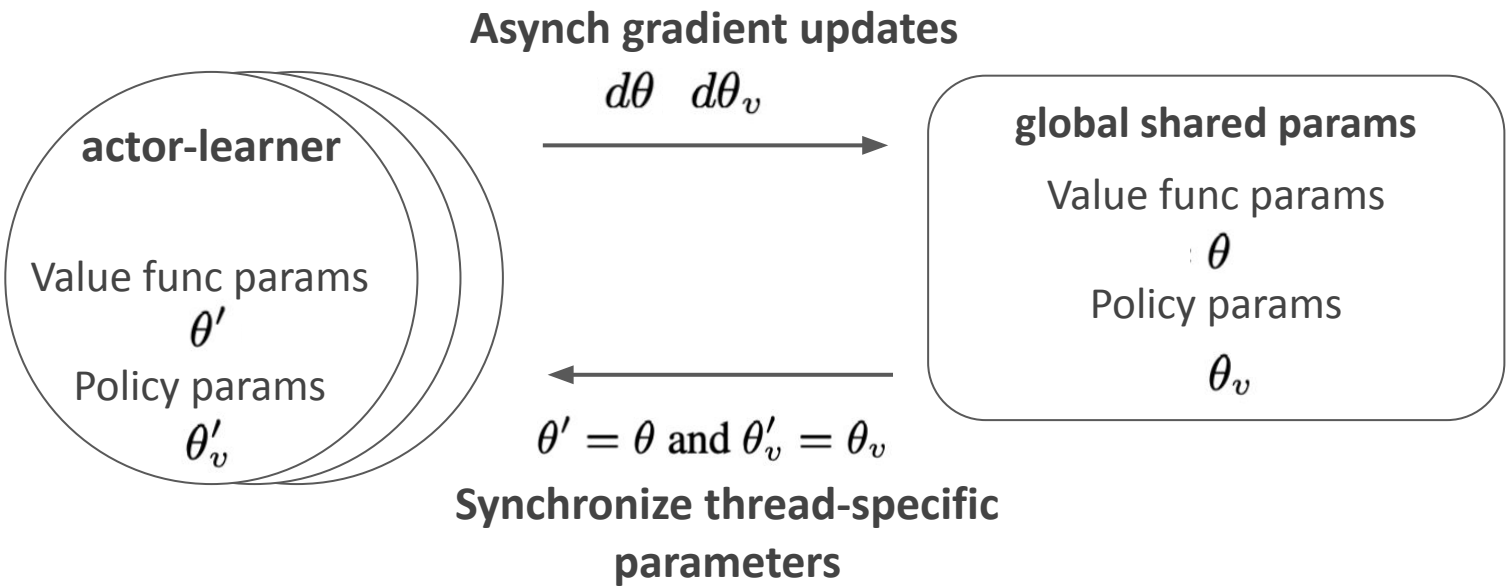
Asynchronous Advantage Actor-Critic (A3C)

- Deep RL algorithms that can train deep neural network policies reliably and without large resource requirements (like GPU)
- Main features:
  - **Asynchronous gradient descent*** for optimization of deep neural network
  - **Multiple actor-learners** (CPU threads) in parallel explore different parts of the environment and perform gradient updates independently and asynchronously

Mnih, V., Badia, A.P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., Silver, D. & Kavukcuoglu, K.. (2016). Asynchronous Methods for Deep Reinforcement Learning. Proceedings of The 33rd International Conference on Machine Learning, 48:1928-1937 Available from https://proceedings.mlr.press/v48/mniha16.html

* Recht, B., Re, C., Wright, S., & Niu, F. (2011). Hogwild!: A lock-free approach to parallelizing stochastic gradient descent. Advances in neural information processing systems, 24.

Asynchronous Advantage Actor-Critic (A3C)



Mnih, V., Badia, A.P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., Silver, D. & Kavukcuoglu, K.. (2016). Asynchronous Methods for Deep Reinforcement Learning. Proceedings of The 33rd International Conference on Machine Learning, 48:1928-1937 Available from https://proceedings.mlr.press/v48/mniha16.html

Asynchronous Advantage Actor-Critic (A3C)

Algorithm S3 Asynchronous advantage actor-critic - pseudocode for each actor-learner thread.

// Assume global shared parameter vectors θ *and* θ_v *and global shared counter* $T = 0$
// Assume thread-specific parameter vectors θ' *and* θ'_v
Initialize thread step counter $t \leftarrow 1$
repeat
  Reset gradients: $d\theta \leftarrow 0$ and $d\theta_v \leftarrow 0$.
  Synchronize thread-specific parameters $\theta' = \theta$ and $\theta'_v = \theta_v$
  $t_{start} = t$
  Get state s_t
  repeat
    Perform a_t according to policy $\pi(a_t|s_t;\theta')$
    Receive reward r_t and new state s_{t+1}
    $t \leftarrow t + 1$
    $T \leftarrow T + 1$
  until terminal s_t **or** $t - t_{start} == t_{max}$
  $R = \begin{cases} 0 & \text{for terminal } s_t \\ V(s_t, \theta'_v) & \text{for non-terminal } s_t \text{// Bootstrap from last state} \end{cases}$
  for $i \in \{t-1, \ldots, t_{start}\}$ **do**
    $R \leftarrow r_i + \gamma R$
    Accumulate gradients wrt θ': $d\theta \leftarrow d\theta + \nabla_{\theta'} \log \pi(a_i|s_i;\theta')(R - V(s_i;\theta'_v))$
    Accumulate gradients wrt θ'_v: $d\theta_v \leftarrow d\theta_v + \partial \left(R - V(s_i;\theta'_v)\right)^2 / \partial \theta'_v$
  end for
  Perform asynchronous update of θ using $d\theta$ and of θ_v using $d\theta_v$.
until $T > T_{max}$

Advantage function
A(s,a) = Q(s,a) - V(s)

In practice, Q(s,a) is replaced with a bootstrap sample

Mnih, V., Badia, A.P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., Silver, D. & Kavukcuoglu, K.. (2016). Asynchronous Methods for Deep Reinforcement Learning. Proceedings of The 33rd International Conference on Machine Learning, 48:1928-1937 Available from https://proceedings.mlr.press/v48/mniha16.html

Asynchronous Advantage Actor-Critic (A3C)

- Actor-learners can have different exploration policies
- By running different exploration policies in different threads, the overall changes being made to the parameters by multiple actor-learners applying online updates in parallel are likely to be **less correlated in time**
  - We do not use a replay memory and rely on parallel
  - We are able to use on-policy reinforcement learning methods (since formally the replay buffer requires an off-policy method)
- Multiple parallel actor-learners reduce training time that is roughly linear in the number of parallel actor-learners

Mnih, V., Badia, A.P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., Silver, D. & Kavukcuoglu, K.. (2016). Asynchronous Methods for Deep Reinforcement Learning. Proceedings of The 33rd International Conference on Machine Learning, 48:1928-1937 Available from https://proceedings.mlr.press/v48/mniha16.html

Asynchronous Advantage Actor-Critic (A3C)

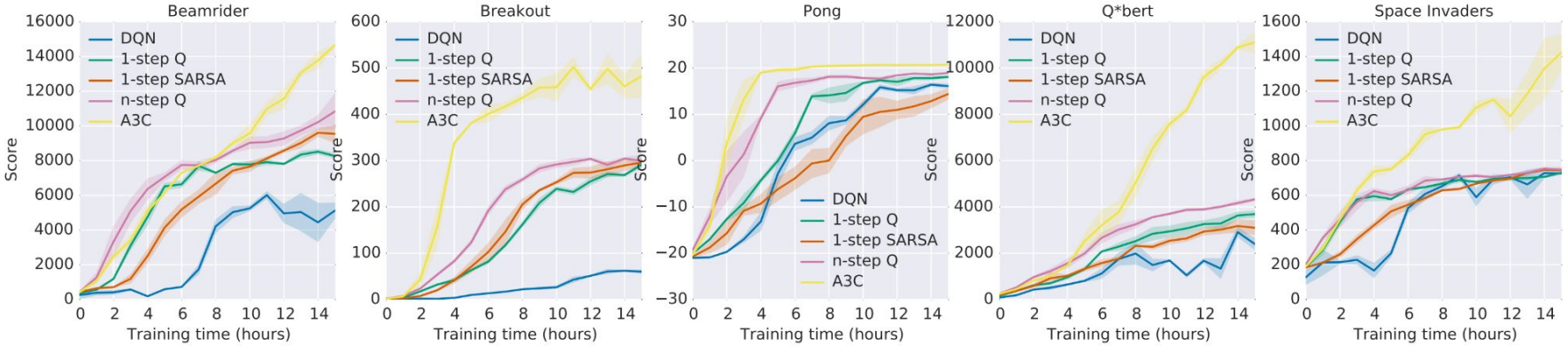


Figure 1. Learning speed comparison for DQN and the new asynchronous algorithms on five Atari 2600 games. DQN was trained on a single Nvidia K40 GPU while the asynchronous methods were trained using 16 CPU cores. The plots are averaged over 5 runs. In the case of DQN the runs were for different seeds with fixed hyperparameters. For asynchronous methods we average over the best 5 models from 50 experiments with learning rates sampled from $LogUniform(10^{-4}, 10^{-2})$ and all other hyperparameters fixed.

Mnih, V., Badia, A.P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., Silver, D. & Kavukcuoglu, K.. (2016). Asynchronous Methods for Deep Reinforcement Learning. Proceedings of The 33rd International Conference on Machine Learning, 48:1928-1937 Available from https://proceedings.mlr.press/v48/mniha16.html

Trust Region Policy Optimization (TRPO)

- In the normal policy gradient, we needed to choose a (very small) step size to keep new and old policies close in parameter space
- Why? Even seemingly small differences in parameter space can have very large differences in performance
- However, this sacrifice the sample efficiency
- TRPO updates policies by taking the largest step possible to improve performance, while satisfying a special constraint on how close the new and old policies are allowed to be
- The constraint is expressed in terms of KL-Divergence, a measure of (something like, but not exactly) distance between probability distributions

https://spinningup.openai.com/en/latest/algorithms/trpo.html

Trust Region Policy Optimization (TRPO)

- Two key concepts
  - Surrogate advantage (a measure of how the current policy θ performs relative to the old policy θ_k)

$$\mathcal{L}(\theta_k, \theta) = \underset{s,a \sim \pi_{\theta_k}}{\mathrm{E}} \left[\frac{\pi_\theta(a|s)}{\pi_{\theta_k}(a|s)} A^{\pi_{\theta_k}}(s,a) \right]$$

  - KL-divergence (a measure of how one probability distribution is different from a second)

$$\bar{D}_{KL}(\theta||\theta_k) = \underset{s \sim \pi_{\theta_k}}{\mathrm{E}} \left[D_{KL}\left(\pi_\theta(\cdot|s)||\pi_{\theta_k}(\cdot|s)\right) \right]$$

- The theoretical TRPO update

$$\theta_{k+1} = \arg\max_\theta \mathcal{L}(\theta_k, \theta) \quad \text{s.t. } \bar{D}_{KL}(\theta||\theta_k) \leq \delta$$

https://spinningup.openai.com/en/latest/algorithms/trpo.html

Surrogate Advantage

$$\mathcal{L}(\theta_k, \theta) = \underset{s,a \sim \pi_{\theta_k}}{\mathrm{E}} \left[\frac{\pi_\theta(a|s)}{\pi_{\theta_k}(a|s)} A^{\pi_{\theta_k}}(s,a) \right]$$

- The probability of taking action a at state s in the current policy divided by the previous one
  - If > 1, the current policy chooses this action a more than the previous policy
  - If < 1, the current policy less likely chooses this action a
- A measure of the difference between two policies

Surrogate Advantage

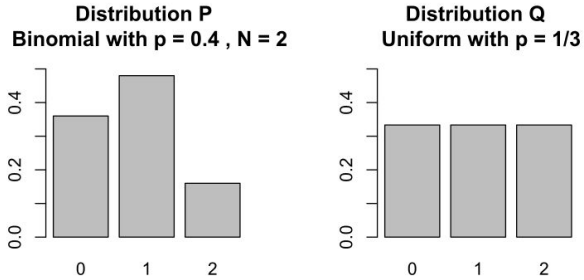
$$\mathcal{L}(\theta_k, \theta) = \underset{s,a \sim \pi_{\theta_k}}{\mathrm{E}} \left[\frac{\pi_\theta(a|s)}{\pi_{\theta_k}(a|s)} A^{\pi_{\theta_k}}(s,a) \right]$$

- The advantage function
  - A(s,a) = Q(s,a) - V(s,a)
- A(s,a) indicates how good this specific action a, compared to the average performance of all actions possible at state s
  - If > 0, action a is better than the average
- Maximizing this ensures to improve the policy with a largest step

Milestone Questions: KL Divergence

- P is a binomial distribution with p = 0.4 and N = 2
- Q is a discrete uniform distribution with the three possible outcomes with p = 1/3
- Compute KL-divergence of P || Q and Q || P

$$D_{\mathrm{KL}}(P \parallel Q) = \sum_{x \in \mathcal{X}} P(x) \ln\left(\frac{P(x)}{Q(x)}\right)$$



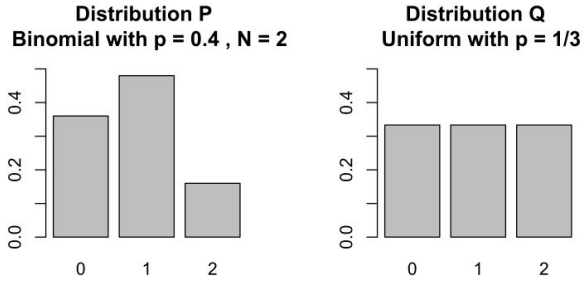
x	0	1	2
Distribution $P(x)$	$\frac{9}{25}$	$\frac{12}{25}$	$\frac{4}{25}$
Distribution $Q(x)$	$\frac{1}{3}$	$\frac{1}{3}$	$\frac{1}{3}$

https://en.wikipedia.org/wiki/Kullback%E2%80%93Leibler_divergence

Milestone Questions: KL Divergence

- P is a binomial distribution with p = 0.4 and N = 2
- Q is a discrete uniform distribution with the three possible outcomes with p = 1/3
- Compute KL-divergence of P || Q and Q || P

$$
\begin{aligned}
D_{\mathrm{KL}}(P \parallel Q) &= \sum_{x \in \mathcal{X}} P(x) \ln\left(\frac{P(x)}{Q(x)}\right) \\
&= \frac{9}{25} \ln\left(\frac{9/25}{1/3}\right) + \frac{12}{25} \ln\left(\frac{12/25}{1/3}\right) + \frac{4}{25} \ln\left(\frac{4/25}{1/3}\right) \\
&= \frac{1}{25}\left(32\ln(2) + 55\ln(3) - 50\ln(5)\right) \approx 0.0852996,
\end{aligned}
$$



x	0	1	2
Distribution $P(x)$	$\frac{9}{25}$	$\frac{12}{25}$	$\frac{4}{25}$
Distribution $Q(x)$	$\frac{1}{3}$	$\frac{1}{3}$	$\frac{1}{3}$

https://en.wikipedia.org/wiki/Kullback%E2%80%93Leibler_divergence

Proximal Policy Optimization (PPO)

- PPO is motivated by the same question as TRPO: how can we take the biggest possible improvement step on a policy using the data we currently have, without stepping so far that we accidentally cause performance collapse?
  - TRPO: a complex second-order method
  - PPO: a family of first-order methods
- PPO methods are significantly simpler to implement, and empirically seem to perform at least as well as TRPO

https://spinningup.openai.com/en/latest/algorithms/ppo.html

Proximal Policy Optimization (PPO)

- PPO-Clip doesn't have a KL-divergence term in the objective (as a constraint)
- Instead, PPO uses a ratio that will indicates the difference between the current and old policies and clip this ratio from a specific range $[1-\epsilon, 1+\epsilon]$
- PPO-Clip update

$$\theta_{k+1} = \arg\max_{\theta} \underset{s,a \sim \pi\theta_k}{\mathrm{E}} \left[L\left(s, a, \theta_k, \theta\right) \right]$$

$$L\left(s, a, \theta_k, \theta\right) = \min\left(\frac{\pi_\theta(a \mid s)}{\pi_{\theta_k}(a \mid s)} A^{\pi_{\theta_k}}(s,a), \quad g\left(\epsilon, A^{\pi_{\theta_k}}(s,a)\right) \right)$$

$$g(\epsilon, A) = \begin{cases} (1+\epsilon)A & A \geq 0 \\ (1-\epsilon)A & A < 0 \end{cases}$$

https://spinningup.openai.com/en/latest/algorithms/ppo.html

Proximal Policy Optimization (PPO)

- When the advantage is positive

$$L\left(s, a, \theta_k, \theta\right) = \min\left(\frac{\pi_\theta(a \mid s)}{\pi_{\theta_k}(a \mid s)}, (1+\epsilon)\right) A^{\pi_{\theta_k}}(s, a)$$

- Compare the policy deviation ratio and the upper bound defined by the hyperparam epsilon
- Choose the smaller update (thus, bounded by the upper ceiling)

https://spinningup.openai.com/en/latest/algorithms/ppo.html

Proximal Policy Optimization (PPO)

- When the advantage is negative

$$L\left(s, a, \theta_k, \theta\right) = \max\left(\frac{\pi_\theta(a \mid s)}{\pi_{\theta_k}(a \mid s)}, (1-\epsilon)\right) A^{\pi_{\theta_k}}(s, a)$$

- Compare the policy deviation ratio and the lower bound defined by the hyperparam epsilon
- Choose the smaller update (thus, bounded by the lower floor)

https://spinningup.openai.com/en/latest/algorithms/ppo.html

Proximal Policy Optimization (PPO)

Algorithm 1 PPO-Clip

1: Input: initial policy parameters θ_0, initial value function parameters ϕ_0
2: **for** $k = 0, 1, 2, \ldots$ **do**
3: Collect set of trajectories $\mathcal{D}_k = \{\tau_i\}$ by running policy $\pi_k = \pi(\theta_k)$ in the environment.
4: Compute rewards-to-go $\hat{R}_t$.
5: Compute advantage estimates, $\hat{A}_t$ (using any method of advantage estimation) based on the current value function V_{ϕ_k}.
6: Update the policy by maximizing the PPO-Clip objective:

$$\theta_{k+1} = \arg\max_{\theta} \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^{T} \min\left(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)} A^{\pi_{\theta_k}}(s_t, a_t), \quad g(\epsilon, A^{\pi_{\theta_k}}(s_t, a_t)) \right),$$

typically via stochastic gradient ascent with Adam.
7: Fit value function by regression on mean-squared error:

$$\phi_{k+1} = \arg\min_{\phi} \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^{T} \left(V_\phi(s_t) - \hat{R}_t \right)^2,$$

typically via some gradient descent algorithm.
8: **end for**

https://spinningup.openai.com/en/latest/algorithms/ppo.html
Implementation: https://pytorch.org/tutorials/intermediate/reinforcement_ppo.html